

BEEE UNIT - 6

Syllabus: Overview of Number Systems, Logic gates including Universal Gates, BCD codes, Excess-3 code, Gray code, Hamming code. Boolean Algebra, Basic Theorems and properties of Boolean Algebra, Truth Tables and Functionality of Logic Gates – NOT, OR, AND, NOR, NAND, XOR and XNOR. Simple combinational circuits–Half and Full Adders. Introduction to sequential circuits, Flip flops, Registers and counters (Elementary Treatment only)

OLD PAPER QUESTIONS:

Short Questions:

1. Where do you apply logic gates?
2. What is the purpose of using Excess-3 code in digital systems?
3. Write the truth-table for AND gate.
4. Write the truth-table for NOT gate.
5. Define the functionality of a NOT gate.
6. What is a flip- flop in digital electronics?

Long Questions:

1. Write truth tables of gates AND, OR, NOR, NAND?
2. Explain in Truth Tables and Functionality of NAND Gate.
3. Compare between logic gates and universal gates in digital electronics prospective?
4. Explain in detail about Gray code code.
5. Discuss about BCD code.
6. Explain in detail about Excess-3 code
7. Discuss about Hamming code.
8. Analyse full adder and half adder combinational circuits?
9. Draw the Logic diagram of Half-adder circuit using basic gates. Explain its operation using Truth table.
10. Discuss in detail about Full adder.
11. Discuss in detail about Flip flops
12. Explain in detail about Registers and counters
13. Give the introduction to sequential circuits, focus on registers and counters?
14. Implement a XOR gate logic function using only NAND gates
15. What is the largest decimal value that can be represented by
16. a 8-bit binary, (b) a 16 bit binary number?

Short Questions:

1. Where do you apply logic gates?

Ans: Logic gates are applied in many electronic and digital systems such as:

Computers and Laptops	Communication Systems
Digital Circuits	Embedded Systems and Microcontrollers
Mobile Phones	Arithmetic and Logical Operations
Automation Systems	

2. What are the logic gates?

Ans: Logic gates are the basic building blocks of digital electronics. They perform logical operations on one or more input signals to produce a single output.

Common logic gates are:

1. **AND Gate** – Output is HIGH only when all inputs are HIGH.
2. **OR Gate** – Output is HIGH when at least one input is HIGH.
3. **NOT Gate** – Gives the opposite (complement) of the input.
4. **NAND Gate** – Opposite of AND gate output.
5. **NOR Gate** – Opposite of OR gate output.
6. **XOR (Exclusive-OR) Gate** – Output is HIGH when inputs are different.
7. **XNOR (Exclusive-NOR) Gate** – Output is HIGH when inputs are same.

3. What are the universal gates?

Ans: The universal gates are:

1. **NAND Gate**
2. **NOR Gate**

These are called universal gates because any logic gate or digital circuit can be constructed using only NAND gates or only NOR gates.

4. What are non-weighted codes?

Ans: Non-weighted codes are binary codes in which the positions of bits do not have fixed positional weights.

Examples of non-weighted codes are:

1. **Gray Code**
2. **Excess-3 (XS-3) Code**
3. **Hamming Code**

These codes are mainly used for error detection, error correction, and reducing errors in digital systems.

5. Define the functionality of a NOT gate.

Ans: A NOT gate is a logic gate that produces the complement of the input signal.

- If the input is 1 (HIGH), the output becomes 0 (LOW).
- If the input is 0 (LOW), the output becomes 1 (HIGH).

It is also called an inverter.

6. What is the purpose of using Excess-3 code in digital systems?

Ans: The purpose of using Excess-3 (XS-3) code in digital systems is to **simplify decimal arithmetic operations and reduce errors**. It is a non-weighted, self-complementing code mainly used in digital circuits for decimal number representation.

7. What is a flip-flop in digital electronics?

Ans:

- ✓ **A flip-flop is a bistable digital circuit used to store one bit of binary data (0 or 1).**
- ✓ It has two stable states and can change its state according to the input signals and clock pulse.
- ✓ Flip-flops are used in registers, counters, memory devices, and sequential circuits.

8. Convert $(12)_8$ in to Decimal Number.

Ans:

$$(12)_8 = 1 \times 8^1 + 2 \times 8^0 = 10(\text{decimal})$$

9. Convert $(25)_{10}$ in to Binary Number.

Ans:

This method uses repeated division by 2
 $25/2 = 12 + \text{remainder } 1$ (Least significant bit)
 $12/2 = 6 + \text{remainder } 0$
 $6/2 = 3 + \text{remainder } 0$
 $3/2 = 1 + \text{remainder } 1$
 $1/2 = 0 + \text{remainder } 1$ (Most significant bit)
Result: $(25)_{10} = 11001$

10. Convert $(547)_8$ in to Hexa Decimal Number.

Ans:

$$\begin{aligned}(547)_8 &= (101 \ 100 \ 111) \text{ (Binary) Step 1} \\ &= (0001) (0110) (0111) \text{ Step 2} \\ &= (167) \text{ (Hexadecimal) Step 3}\end{aligned}$$

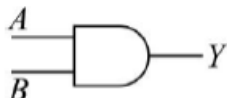
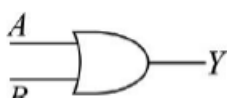
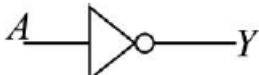
Long Questions:

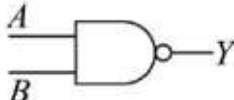
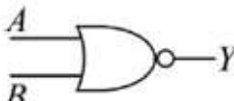
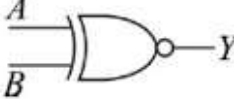
1. Write truth tables of gates AND, OR, NOT, NAND, NOR, XOR, XNOR?

Ans: Logic gates are the basic building blocks of digital electronics. They perform logical operations on one or more input signals to produce a single output.

Common logic gates are:

1. **AND Gate** – Output is HIGH only when all inputs are HIGH.
2. **OR Gate** – Output is HIGH when at least one input is HIGH.
3. **NOT Gate** – Gives the opposite (complement) of the input.
4. **NAND Gate** – Opposite of AND gate output.
5. **NOR Gate** – Opposite of OR gate output.
6. **XOR (Exclusive-OR) Gate** – Output is HIGH when inputs are different.
7. **XNOR (Exclusive-NOR) Gate** – Output is HIGH when inputs are same.

S.No.	Gate	Logic Symbol	Function	Truth table															
1	AND		$Y = A \cdot B$	<table><tr><td>A</td><td>B</td><td>Y</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	1	0	0	0	1	0	1	1	1
A	B	Y																	
0	0	0																	
1	0	0																	
0	1	0																	
1	1	1																	
2	OR		$Y = A + B$	<table><tr><td>A</td><td>B</td><td>Y</td></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	1	0	1	0	1	1	1	1	1
A	B	Y																	
0	0	0																	
1	0	1																	
0	1	1																	
1	1	1																	
3	NOT		$Y = \overline{A}$	<table><tr><td>A</td><td>Y</td></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0									
A	Y																		
0	1																		
1	0																		

4	NAND		$Y = \overline{A \cdot B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	1	0	1	0	1	1	1	1	0
A	B	Y																	
0	0	1																	
1	0	1																	
0	1	1																	
1	1	0																	
5	NOR		$Y = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	1	0	0	0	1	0	1	1	0
A	B	Y																	
0	0	1																	
1	0	0																	
0	1	0																	
1	1	0																	
6	XOR		$Y = A \oplus B$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	1	0	1	0	1	1	1	1	0
A	B	Y																	
0	0	0																	
1	0	1																	
0	1	1																	
1	1	0																	
7	XNOR		$Y = \overline{A \oplus B}$	<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	1	1	0	0	0	1	0	1	1	1
A	B	Y																	
0	0	1																	
1	0	0																	
0	1	0																	
1	1	1																	

2. Compare between logic gates and universal gates in digital electronics prospective?

Ans:

Logic Gates	Universal Gates
Logic gates are basic digital circuits used to perform logical operations.	Universal gates are special logic gates that can implement any Boolean function.
Examples: AND, OR, NOT, XOR, XNOR, NAND, NOR.	Examples: NAND and NOR gates only.

Logic Gates	Universal Gates
Different gates are required for different operations.	A single type of universal gate can be used to build all other gates.
Used for specific logical functions in digital circuits.	Used for designing complete digital systems using one gate type.
Circuit design may require multiple gate types.	Simplifies fabrication and reduces cost in digital electronics.

3. Discuss about BCD code.

Ans: BCD (Binary Coded Decimal) is a weighted binary code used to represent decimal numbers in binary form. Each digit of a decimal number is replaced by a 4-bit binary number. Since a decimal digit is a number from 0 to 9, a nibble representing a number greater than 9 is invalid BCD.

	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Example: Decimal number 25 is represented in BCD as:

$$2 = 0010, 5 = 0101$$

So, BCD of 25 = 0010 0101

Features of BCD Code:

1. It is a weighted code with weights 8, 4, 2, and 1.
2. Each decimal digit is encoded separately into 4 bits.
3. Only binary numbers from 0000 to 1001 are valid in BCD.

Advantages:

1. Easy conversion between decimal and binary numbers.
2. Widely used in digital clocks, calculators, and digital displays.
3. Simple to display decimal digits.

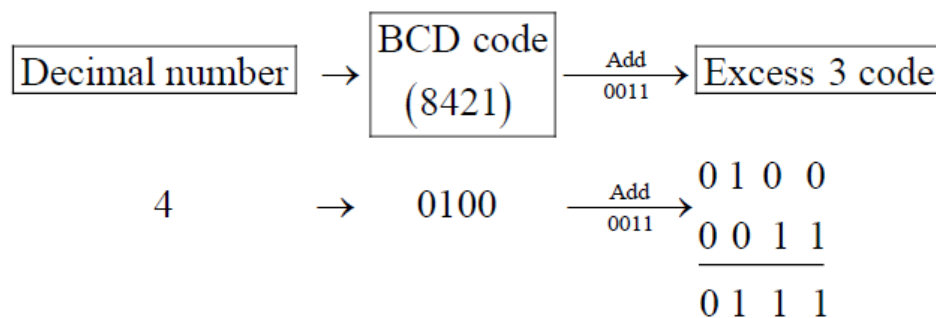
Disadvantages:

1. Requires more bits compared to pure binary representation.
2. Arithmetic operations are more complex.

4. Explain in detail about Excess-3 code

Ans:

- Excess-3 is a non-weighted binary code used to represent decimal digits. It is mainly used in digital systems for simplifying arithmetic operations and reducing errors.
- It is a way to represent values with a balanced number of positive and negative numbers using a pre-specified number 3 as a biasing value.
- The Excess-3 is derived from 8421 BCD code by adding binary value (0011)₂ to each code in 8421.



The relationship between the four bit binary codes & Excess -3 codes are

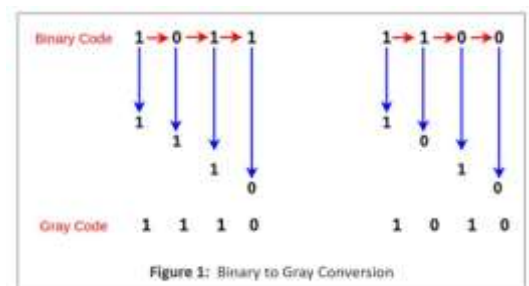
Decimal	BCD code 8421	Excess-3 code
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

5. Explain in detail about Gray code.

Ans: Gray Code: Gray code is a non-weighted binary code in which only one-bit changes at a time when moving from one code to the next. It is also called a unit distance code, cyclic code or reflected code.

Why Gray Code is Used?

- ✓ Reduces errors during transitions
- ✓ Minimizes switching changes
- ✓ Used in digital systems, ADCs, encoders



XOR		
A	B	$O = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Example: Convert $(1010)_2$ into gray code.

$$\begin{array}{cccc} 1 & 0 & 1 & 0 \\ \downarrow & & & \\ 1 & 1 & 1 & 1 \end{array}$$
 $1010_2 = 1111_{\text{gray}}$

Example: Convert $(0110)_2$ into gray code.

$$\begin{array}{cccc} 0 & 1 & 1 & 0 \\ \downarrow & & & \\ 0 & 1 & 0 & 1 \end{array}$$
 $0110_2 = 0101_{\text{gray}}$

Features of Gray Code:

1. It is a non-weighted code.
2. Only one bit changes between successive numbers.
3. Reduces errors during data transmission and switching operations.
4. It is not suitable for direct arithmetic operations.

Advantages:

1. Minimizes errors in digital systems.
2. Used in position encoders and analog to digital converters.
3. Reduces switching noise in electronic circuits.

Disadvantages:

1. Arithmetic operations are difficult.
2. Conversion to binary is required for calculations

6. Discuss about Hamming code.

Ans: Hamming code:

- ✓ Hamming code is a method used for error detection and correction in digital communication and storage.
- ✓ It was developed by Richard Hamming in the 1950s.
- ✓ The primary goal of Hamming code is to detect and correct single-bit errors that may occur during data transmission.
- ✓ Used for single bit error detection & correction.
- ✓ Using parity data, error of the data is detected.
- ✓ Easy to implement.

- ✓ In Hamming code, the source encodes the message by adding parity bits in the message. These parity bits are inserted and generated at Certain positions in the message to accomplish Correction process.

Step1: Calculation of total number of parity bits:

Let us assume that the message contains

m = number of message bits

p = number of parity bits

Number of parity bits can be calculated by using the formula

where $m + p$ = total number of bits

Step2: Placing the parity bits in their correct position:

p -parity bits should be placed at bit positions of powers of 2

P_4	m_4	m_3	m_2	P_3	m_1	P_2	P_1
8	7	6	5	4	3	2	1
2^3				2^2		2^1	2^0

Step 3: Calculating the values of parity bits:

The parity bits make the number of 1's either even or odd.

Odd parity: Total number of bits in the message is made odd is called odd parity

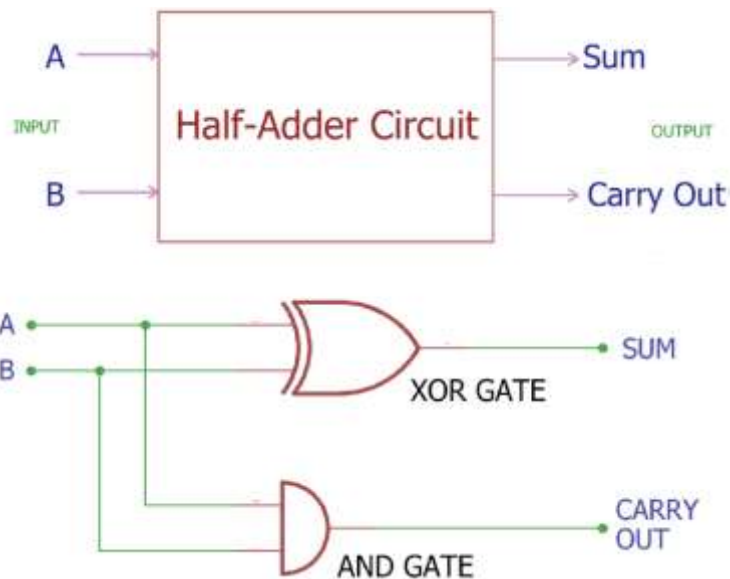
Even parity: Total number of 1's is even.

7. Analyse full adder and half adder combinational circuits?

Ans: A Half Adder and a Full Adder are fundamental building blocks of digital electronics used to perform binary addition. They form the basis of arithmetic operations inside computers and digital systems.

Half Adder: A Half Adder is a combinational logic circuit used to add two binary inputs A and B. It is designed using one EX-OR gate and one AND gate. The circuit produces two outputs: Sum and Carry.

The output obtained from the EX-OR gate is the sum of the two numbers while that obtained by AND gate is the carry. There will be no forwarding of carry addition because there is no logic gate to process that. Thus, this is called the Half Adder circuit.



Logical Expression of Half Adder: The Logical Expression for half added is given as

$$Sum = A \oplus B$$

$$Carry = AB$$

Truth Table of Half Adder

The Truth Table for Half Added is Given as

Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

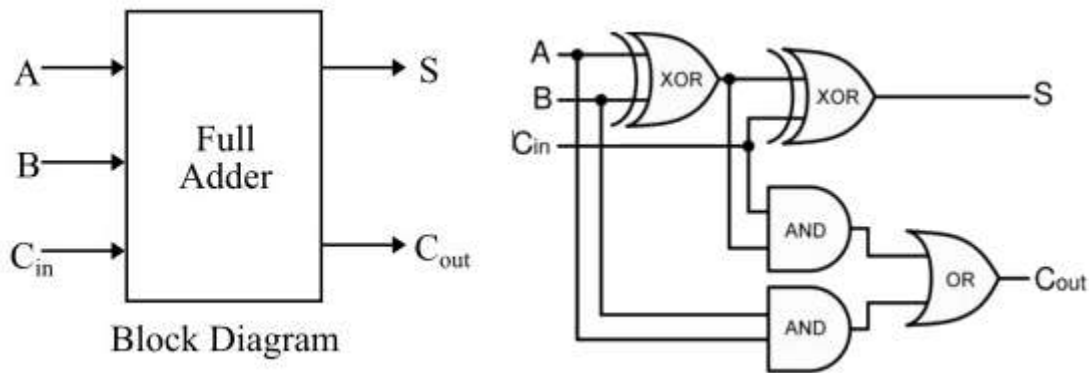
Advantages:

- Flexible and easy when it comes to design.
- Involves the use of fewer logic gates thus, is cheaper.

Disadvantages:

- Fails to process a carry input from the previously added numbers.
- Restricted to the addition of only two bits.

Full Adder: A Full Adder is a combinational circuit used to add three binary inputs: A, B, and carry input (C_{in}). It produces two outputs: Sum (S) and Carry output (C_{out}). A full adder consists of two EX-OR gates, two AND gates, and one OR gate.



The equation obtained by the EX-OR gate is the sum of the binary digits. While the output obtained by AND gate is the carry obtained by addition.

Logical Expression of Full Adder: Given Below is the Logical Expression of Full Adder

$$SUM = (A \oplus B) \oplus C_{in}$$

$$CARRY-OUT = A.B + C_{in}(A \oplus B)$$

Truth Table of Full Adder:

Input			Output	
A	B	C _{in}	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Advantages:

- Can add 3 bits, it includes one carry input and a carry output, which can perform more elaborate computations.
- It can be cascaded to produce adders for a number of bit additions which makes it suitable for multi bit arithmetic.

Disadvantages:

- Complex and needs more gates, hence making the design more complicate and expensive.
- Slightly slower because normally 2 gate process are used instead of 1.

8. Discuss in detail about Flip flops.

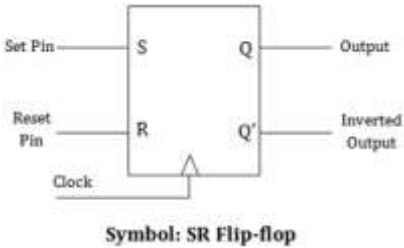
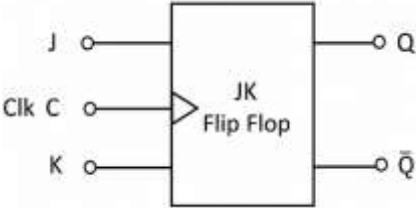
Ans: **Flip-Flops:** A flip-flop is a bistable sequential circuit used to store one bit of binary information (0 or 1). It has two stable states and changes its state according to input signals and clock pulses.

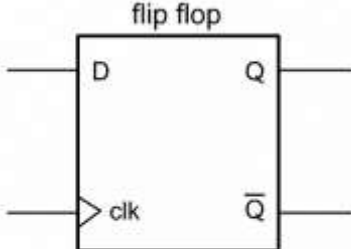
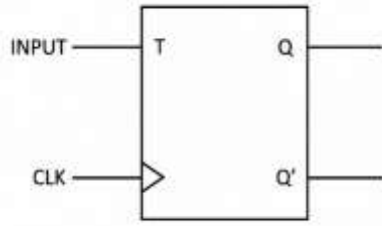
Flip-flops are basic memory elements in digital electronics and are widely used in sequential circuits.

Characteristics of Flip-Flops

1. Stores one bit of data.
2. Works with clock signals.
3. Has two outputs:
 - Normal output (Q)
 - Complement output ($\bar{Q}$)
4. Used in memory and timing circuits.

Types of Flip-Flops:

Type of Flip Flop	Block diagram	Output															
SR Flip-Flop • Has two inputs: Set (S) and Reset (R). • Used to set or reset the output.	 Symbol: SR Flip-flop	<table><tr><th>S</th><th>R</th><th>Q (Next State)</th></tr><tr><td>0</td><td>0</td><td>No change</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Invalid</td></tr></table>	S	R	Q (Next State)	0	0	No change	0	1	0	1	0	1	1	1	Invalid
S	R	Q (Next State)															
0	0	No change															
0	1	0															
1	0	1															
1	1	Invalid															
JK Flip-Flop • Improved version of SR flip-flop. • Eliminates invalid condition.		<table><tr><th>J</th><th>K</th><th>Q (Next State)</th></tr><tr><td>0</td><td>0</td><td>No change</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>Toggle</td></tr></table>	J	K	Q (Next State)	0	0	No change	0	1	0	1	0	1	1	1	Toggle
J	K	Q (Next State)															
0	0	No change															
0	1	0															
1	0	1															
1	1	Toggle															

D Flip-Flop - Has a single input called Data (D). - Stores input data at clock pulse.		<table><tr><th>D</th><th>Q (Next State)</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	D	Q (Next State)	0	0	1	1
D	Q (Next State)							
0	0							
1	1							
T Flip-Flop Used for toggling operation.		<table><tr><th>T</th><th>Q (Next State)</th></tr><tr><td>0</td><td>No change</td></tr><tr><td>1</td><td>Toggle</td></tr></table>	T	Q (Next State)	0	No change	1	Toggle
T	Q (Next State)							
0	No change							
1	Toggle							

Applications of Flip-Flops

1. Registers and memory devices
2. Counters
3. Frequency dividers
4. Digital clocks
5. Shift registers and control circuits

Thus, flip-flops are important storage elements used in digital systems and sequential logic circuits.

9. Explain in detail about Registers and counters. (or) Give the introduction to sequential circuits, focus on registers and counters?

Ans: Registers: A register is a group of flip-flops used to store binary data temporarily in a digital system. Each flip-flop stores one bit of information. Registers are widely used in computers and digital electronics for data storage and transfer.

Types of Registers

1. Shift Register: A shift register transfers data from one flip-flop to another with each clock pulse.

Types of Shift Registers

- Serial In Serial Out (SISO)

- Serial In Parallel Out (SIPO)
- Parallel In Serial Out (PISO)
- Parallel In Parallel Out (PIPO)

2. Buffer Register: Used for temporary storage of data during processing.

3. Control Register: Stores control information and instructions.

Applications of Registers

1. Temporary data storage
2. Data transfer operations
3. Arithmetic and logic operations
4. Used in microprocessors and digital systems

Counters:

A counter is a sequential circuit that counts clock pulses and gives binary output. Counters are made using flip-flops.

Types of Counters

1. Asynchronous (Ripple) Counter

- Flip-flops are triggered one after another.
- Simple design but slower in operation.

2. Synchronous Counter

- All flip-flops are triggered simultaneously by the same clock pulse.
- Faster than asynchronous counters.

3. Up Counter: Counts in increasing order:

$000 \rightarrow 001 \rightarrow 010 \rightarrow 011$

4. Down Counter: Counts in decreasing order:

$111 \rightarrow 110 \rightarrow 101$

5. Up/Down Counter

Can count in both increasing and decreasing order.

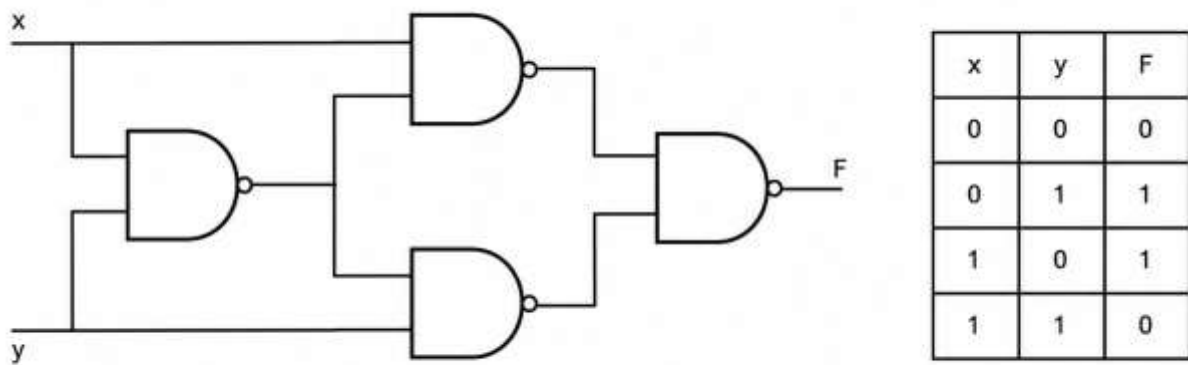
Applications of Counters

1. Digital clocks
2. Frequency counters
3. Event counting
4. Time measurement
5. Traffic light control systems

10. Implement a XOR gate logic function using only NAND and NOR gates.

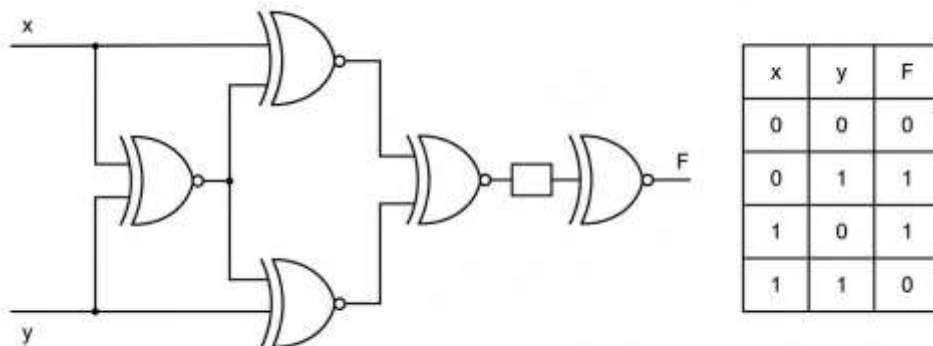
Ans: XOR gate Implementation using NAND Gates: XOR gate can be implemented using four NAND gates as illustrated below. The circuit first detects differences between inputs and processes them to produce the XOR function.

Implementation of XOR gate using four NAND gate as shown in the picture below



XOR gate Implementation using NOR Gates: The XOR gate is implemented by five gates and these gates are the NOR gates. The inputs are then passed through a number of NOR gates to remove all the non-exclusive conditions before the XOR is applied to them.

Implementation of XOR gate using five NOR gate as shown in the picture below:



11. What is the largest decimal value that can be represented by

(a) a 8-bit binary, (b) a 16 bit binary number?

Ans: The largest decimal value that can be represented by an n -bit binary number is:

$$2^n - 1$$

(a) 8-bit binary number

$$2^8 - 1 = 256 - 1 = 255$$

So, the largest decimal value is:

255

(b) 16-bit binary number

$$2^{16} - 1 = 65536 - 1 = 65535$$

So, the largest decimal value is:

65535
